

Reviewer Pack — Minimal Entropy of Random Permutation Matrices and Communica...

Entry 9a01d8e999c6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 09:54:38 UTC

Minimal Entropy of Random Permutation Matrices and Communication Complexity for AND-OR Trees

Entry ID: 9a01d8e999c6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 09:54:38 UTC

1. Conjecture

Field A (mathematical branch): Random Matrix Theory **Field B** (complexity object): Communication Complexity

Statement:

{‘s1’: ‘For any given n , the minimal entropy of a random permutation matrix is asymptotically equal to the communication complexity of an AND-OR tree with n leaves.’, ‘s2’: ‘The entropy of a random permutation matrix is defined as the negative sum of its eigenvalues divided by $\log(2)$.’, ‘s3’: ‘Communication complexity for an AND-OR tree is measured by the maximum number of bits required to transmit all inputs and outputs between parties.’}

Rationale (proposer’s reasoning):

{‘s1’: ‘Random permutation matrices have been studied in the context of quantum information theory, where they are related to unitary operations.’, ‘s2’: ‘Communication complexity captures the inherent difficulty of sharing information between parties, which is a fundamental concept in distributed computing.’, ‘s3’: ‘Linking these two concepts could provide new insights into the nature of computation and communication.’}

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 929609cb67277c0d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for a sample size of $n = 40$ seeds, the mean entropy of random permutation matrices falls within ± 3 bits of the mean communication complexity of AND-OR trees with n leaves, and no seed produces an entropy greater than 10 bits or a communication complexity less than -7 bits.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "random permutation matrices" AND "communication complexity" AND "AND-OR trees" - "entropy of random permutation matrices" AND "communication complexity" AND "AND-OR tree" - "eigenvalues of random permutation matrices" AND "communication complexity" AND "AND-OR tree entropy"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_permutation_matrix(n):
        matrix = [[0] * n for _ in range(n)]
        indices = list(range(n))
        random.shuffle(indices)
        for i, idx in enumerate(indices):
            matrix[i][idx] = 1
        return matrix

    def calculate_entropy(matrix):
        eigenvalues = [matrix[0][i] for i in range(len(matrix))]
        entropy = -sum(e * math.log2(e) for e in eigenvalues if e != 0)
        return entropy

    def generate_and_or_tree(n):
        if n == 1:
            return "leaf"
        else:
            left = generate_and_or_tree(n // 2)
            right = generate_and_or_tree((n + 1) // 2)
            return f"and({left}, {right})"

    def calculate_communication_complexity(tree):
        if tree == "leaf":
```

```

        return 0
    else:
        left, right = tree.split("(")[1].split(",")[0], tree.split(",")[1].strip("(")
        return max(calculate_communication_complexity(left), calculate_communication_complexity(right))

n_values = [5, 10, 15, 20, 30, 40]
total_entropy = 0
total_complexity = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        matrix = generate_permutation_matrix(n)
        entropy = calculate_entropy(matrix)
        tree = generate_and_or_tree(n)
        complexity = calculate_communication_complexity(tree)

        if entropy > 10 or complexity < -7:
            return {
                "metric_name": "Entropy/Complexity",
                "metric_value": None,
                "instances_tested": instances_tested,
                "conjecture_holds": False,
                "counterexample": f"Seed {seed}: Entropy={entropy}, Complexity={complexity}"
            }

        total_entropy += entropy
        total_complexity += complexity
        instances_tested += 1

mean_entropy = total_entropy / instances_tested
mean_complexity = total_complexity / instances_tested

return {
    "metric_name": "Entropy/Complexity",
    "metric_value": {"mean_entropy": mean_entropy, "mean_complexity": mean_complexity},
    "instances_tested": instances_tested,
    "conjecture_holds": abs(mean_entropy - mean_complexity) <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_entropy = sum(r["metric_value"]["mean_entropy"] for r in results if "mean_entropy" in r)
    mean_complexity = sum(r["metric_value"]["mean_complexity"] for r in results if "mean_complexity" in r)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all("mean_entropy" in r and "mean_complexity" in r for r in results):

```

```

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean_entropy={mean_entropy} std_entropy=0 support_fraction={support_fraction}")
    else:
        print(f"RESULT: FALSIFIED counterexample='Support fraction too low' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE Some trials did not produce valid entropy/complexity values.")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73b433d3.py", line 91, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73b433d3.py", line 59, in run_trial
    complexity = calculate_communication_complexity(tree)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73b433d3.py", line 47, in calculate_communication_complexity
    return max(calculate_communication_complexity(left), calculate_communication_complexity(right))
               ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_73b433d3.py", line 46, in calculate_communication_complexity
    left, right = tree.split("(")[1].split(",")[0], tree.split("(")[1].strip(")")
                    ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required calculations to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14239
2	preregistration	ollama_remote	glm4:latest	0	0	9298
3	novelty	ollama_remote	glm4:latest	0	0	8913
4	novelty	ollama_remote	glm4:latest	0	0	8867

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16916
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9082
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9010
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10529
9	judge	ollama_remote	glm4:latest	0	0	11446

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98301 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/9a01d8e999c6.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/9a01d8e999c6.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/9a01d8e999c6.tar.gz* (if generated)